

lec9

April 24, 2026

1 Programming Languages and Environments

1.1 (Lecture 9)

1.1.1 Syllabus

- Exercises on Lists
- Higher-order functions
- Maps and folds

2 Examples

1. Iterating over lists of integers
 1. Membership
 2. Average of elements
 3. Elements greater than
 4. Counting (higher-order)
2. List transformation
 1. Projection
 2. Filter
 3. Zip and unzip
 4. Function composition
3. Compacting elements
 1. Uncompacting
4. Grouping elements
 1. Ungrouping elements
5. Merging sorted lists
6. Merge sort

3 Iterating over lists of integers

3.1 Membership

```
[ ]: (** [mem a l] is true if and only if a is equal to an element of list [l].  
      pre: [true]  
      *)  
let rec mem x l =  
  match l with
```

```
| [] -> false
| y::ys -> if x = y then true else mem x ys
```

```
[ ]: let _ = assert (mem 1 [1;2;3;4;5] = true)
let _ = assert (mem 'a' ['a';'b';'c';'d';'e'] = true)
```

3.2 Computing the average of elements in a list

```
[ ]: (** [avg l] is the average of the elements in list [l].
pre: [length l > 0] is not empty.
*)
let avg l =
  let rec sum l =
    match l with
    | [] -> 0
    | x::xs -> x + sum xs
  in
  let rec len l =
    match l with
    | [] -> 0
    | x::xs -> 1 + len xs
  in
  (sum l) / (len l)
```

```
[ ]: (** [avg l] is the average of the elements in list [l].
pre: [length l > 0] is not empty.
*)
let avg l =
  let rec sum_count l =
    match l with
    | [] -> (0, 0)
    | x::xs ->
      let (sum, count) = sum_count xs in
      (sum + x, count + 1)
  in
  let (sum, count) = sum_count l in sum / count
```

```
[ ]: (** [avg l] is the average of the elements in list [l].
pre: [length l > 0] is not empty.
*)
let avg l =
  let rec sum_count l acc count =
    match l with
    | [] -> (acc, count)
    | x::xs -> sum_count xs (acc + x) (count + 1)
  in
  let (sum, count) = sum_count l 0 0 in sum / count
```

```
[ ]: (** [avg l] is the average of the elements in list [l].
      pre: [length l > 0] is not empty.
      *)
let avg l =
  let (sum, count) = List.fold_left (fun (sum, count) x -> (sum + x, count + 1)) (0, 0) l
  in sum/count
```

```
[ ]: let _ = assert (avg [1;2;3;4;5] = 3)
let _ = assert (avg [1;2;3;4;5;6;7] = 4)
```

3.3 Counting elements greater than some threshold

```
[ ]: (** [count_greater_than x l] is the number of elements in list [l] that are
      ↪greater than [x].
      pre: [true]
      *)
let rec count_greater_than x l =
  match l with
  | [] -> 0
  | y::ys ->
    if y > x then 1 + count_greater_than x ys
    else count_greater_than x ys
```

```
[ ]: let _ = assert (count_greater_than 3 [1;2;3;4;5] = 2)
let _ = assert (count_greater_than 'f' ['a';'b';'c';'d';'e';'f';'g'] = 1)
let _ = assert (count_greater_than 3. [1.;2.;3.;4.;5.] = 2)
let _ = assert (count_greater_than "hello" ["hello";"world";"hello";"world";
      ↪"hello"] = 2)
```

```
[ ]: (** [count_greater_than x l] is the number of elements in list [l] that are
      ↪greater than [x].
      pre: [true]
      *)
let count_greater_than x l =
  List.fold_left (fun n y -> if y > x then n + 1 else n) 0 l
```

```
[ ]: let _ = assert (count_greater_than 3 [1;2;3;4;5] = 2)
let _ = assert (count_greater_than 'f' ['a';'b';'c';'d';'e';'f';'g'] = 1)
let _ = assert (count_greater_than 3. [1.;2.;3.;4.;5.] = 2)
let _ = assert (count_greater_than "hello" ["hello";"world";"hello";"world";
      ↪"hello"] = 2)
```

3.4 Counting elements that satisfy a given predicate (higher-order)

```
[ ]: (** [count_if p l] is the number of elements in list [l] that satisfy predicate
      ↪ [p].
      pre: [true]
      *)
let rec count_if p l =
  match l with
  | [] -> 0
  | x::xs ->
    if p x then 1 + count_if p xs
    else count_if p xs
```

```
[ ]: let count_greater_than n = count_if (fun x -> x > n)

let _ = assert (count_greater_than 3 [1;2;3;4;5] = 2)
let _ = assert (count_greater_than 'f' ['a';'b';'c';'d';'e';'f';'g'] = 1)
let _ = assert (count_greater_than 3. [1.;2.;3.;4.;5.] = 2)
let _ = assert (count_greater_than "hello" ["hello";"world";"hello";"world";
      ↪ "hello"] = 2)
```

```
[ ]: let count_less_than n = count_if (fun x -> x < n)

let _ = assert (count_less_than 3 [1;2;3;4;5] = 2)
let _ = assert (count_less_than 'f' ['a';'b';'c';'d';'e';'f';'g'] = 5)
let _ = assert (count_less_than 3. [1.;2.;3.;4.;5.] = 2)
let _ = assert (count_less_than "hello" ["hello";"world";"hello";"world";
      ↪ "hello"] = 0)
```

```
[ ]: type person = {name: string; age: int}
let count_age_equal_to n = count_if (fun x -> x.age = n)

let _ = assert (count_age_equal_to 3 [{name="a"; age=1}; {name="b"; age=2};
      ↪ {name="c"; age=3}; {name="d"; age=4}; {name="e"; age=3}] = 2)
let _ = assert (count_age_equal_to 1 [{name="a"; age=1}; {name="b"; age=2};
      ↪ {name="c"; age=3}; {name="d"; age=4}; {name="e"; age=5}] = 1)
```

```
[ ]: (** [count_if p l] is the number of elements in list [l] that satisfy predicate
      ↪ [p].
      pre: [true]
      *)
let count_if p l =
  List.fold_left (fun n x -> if p x then n + 1 else n) 0 l
```

```
[ ]: type person = {name: string; age: int}
let count_age_equal_to n = count_if (fun x -> x.age = n)
```

```

let _ = assert (count_age_equal_to 3 [{name="a"; age=1}; {name="b"; age=2}; ␣
  ↪ {name="c"; age=3}; {name="d"; age=4}; {name="e"; age=3}] = 2)
let _ = assert (count_age_equal_to 1 [{name="a"; age=1}; {name="b"; age=2}; ␣
  ↪ {name="c"; age=3}; {name="d"; age=4}; {name="e"; age=5}] = 1)

```

4 List transformation

4.1 Projection

```

[ ]: (** [project_fst l] is the list of the first elements of the pairs in list [l].
      pre: [true]
      *)
let rec project_fst l =
  match l with
  | [] -> []
  | (x,_)::xs -> x::project_fst xs

```

```

[ ]: let _ = assert (project_fst [] = [])
let _ = assert (project_fst [(1,2);(3,4);(5,6)] = [1;3;5])

```

```

[ ]: (** [project_fst l] is the list of the first elements of the pairs in list [l].
      pre: [true]
      *)
let project_fst l =
  List.map (fun (x,_) -> x) l

```

```

[ ]: let _ = assert (project_fst [] = [])
let _ = assert (project_fst [(1,2);(3,4);(5,6)] = [1;3;5])

```

```

[ ]: (** [project_name l] is the list of the names of the people in list [l].
      pre: [true]
      *)
let rec project_name l =
  match l with
  | [] -> []
  | {name=n; age=_}::ps -> n::project_name ps

```

```

[ ]: let _ = assert (project_name [] = [])
let _ = assert (project_name [{name="a"; age=1}; {name="b"; age=2}; {name="c"; ␣
  ↪ age=3}; {name="d"; age=4}; {name="e"; age=5}] = ["a";"b";"c";"d";"e"])

```

```

[ ]: (** [project_name l] is the list of the names of the people in list [l].
      pre: [true]
      *)
let project_name l =
  List.map (fun x -> x.name) l

```

```
[ ]: let _ = assert (project_name [] = [])
let _ = assert (project_name [{name="a"; age=1}; {name="b"; age=2}; {name="c"; age=3}; {name="d"; age=4}; {name="e"; age=5}] = ["a"; "b"; "c"; "d"; "e"])
```

4.2 Filtering

```
[ ]: (** [filter l] is the list of the names of the people in list [l].
pre: [true]
*)
let rec filter p l =
  match l with
  | [] -> []
  | x::xs -> if p x then x::filter p xs else filter p xs
```

```
[ ]: let _ = assert (filter (fun x -> x = "hello") [] = [])
let _ = assert (filter (fun x -> x = "hello") ["hello"; "world"; "hello"; "world"; "hello"] = ["hello"; "hello"; "hello"])
let _ = assert (filter (fun x -> x > 3) [1;2;3;4;5] = [4;5])
let _ = assert (filter (fun x -> x < 'f') ['a'; 'b'; 'c'; 'd'; 'e'; 'f'; 'g'] = ['a'; 'b'; 'c'; 'd'; 'e'])
```

```
[ ]: (** [filter l] is the list of the names of the people in list [l].
pre: [true]
*)
let filter p l =
  List.fold_left (fun acc x -> if p x then acc@[x] else acc) [] l
```

```
[ ]: let _ = assert (filter (fun x -> x = "hello") [] = [])
let _ = assert (filter (fun x -> x = "hello") ["hello"; "world"; "hello"; "world"; "hello"] = ["hello"; "hello"; "hello"])
let _ = assert (filter (fun x -> x > 3) [1;2;3;4;5] = [4;5])
let _ = assert (filter (fun x -> x < 'f') ['a'; 'b'; 'c'; 'd'; 'e'; 'f'; 'g'] = ['a'; 'b'; 'c'; 'd'; 'e'])
```

```
[ ]: (** [filter l] is the list of the names of the people in list [l].
pre: [true]
*)
let filter p l =
  List.rev (List.fold_left (fun acc x -> if p x then x::acc else acc) [] l)
```

```
[ ]: let _ = assert (filter (fun x -> x = "hello") [] = [])
let _ = assert (filter (fun x -> x = "hello") ["hello"; "world"; "hello"; "world"; "hello"] = ["hello"; "hello"; "hello"])
let _ = assert (filter (fun x -> x > 3) [1;2;3;4;5] = [4;5])
let _ = assert (filter (fun x -> x < 'f') ['a'; 'b'; 'c'; 'd'; 'e'; 'f'; 'g'] = ['a'; 'b'; 'c'; 'd'; 'e'])
```

```
[ ]: (** [filter l] is the list of the names of the people in list [l].
      pre: [true]
      *)
let filter p l =
  List.fold_right (fun x acc -> if p x then x::acc else acc) l []
```

```
[ ]: let _ = assert (filter (fun x -> x = "hello") [] = [])
let _ = assert (filter (fun x -> x = "hello") ["hello";"world";"hello";"world";
  ↪ "hello"] = ["hello";"hello";"hello"])
let _ = assert (filter (fun x -> x > 3) [1;2;3;4;5] = [4;5])
let _ = assert (filter (fun x -> x < 'f') ['a';'b';'c';'d';'e';'f';'g'] = ['a';
  ↪ 'b';'c';'d';'e'])
```

```
[ ]: (** [filter l] is the list of the names of the people in list [l].
      pre: [true]
      *)
let filter p l =
  List.fold_right (fun x xs -> x @ xs) (List.map (fun x -> if p x then [x] else
  ↪ []) l) []
```

```
[ ]: let _ = assert (filter (fun x -> x = "hello") [] = [])
let _ = assert (filter (fun x -> x = "hello") ["hello";"world";"hello";"world";
  ↪ "hello"] = ["hello";"hello";"hello"])
let _ = assert (filter (fun x -> x > 3) [1;2;3;4;5] = [4;5])
let _ = assert (filter (fun x -> x < 'f') ['a';'b';'c';'d';'e';'f';'g'] = ['a';
  ↪ 'b';'c';'d';'e'])
```

Is there a version better than other? In what situation is it better to use one? What are the arguments to consider?

4.3 Zip and Unzip

```
[ ]: (** [zip l1 l2] is the list of pairs of elements from lists [l1] and [l2].
      pre: [length l1 = length l2]
      *)
let rec zip l1 l2 =
  match (l1, l2) with
  | ([], []) -> []
  | (x::xs, y::ys) -> (x,y)::zip xs ys
  | (_, _) -> failwith "zip: lists have different lengths"
```

```
[ ]: let _ = assert (zip [1;2;3] ['a';'b';'c'] = [(1,'a');(2,'b');(3,'c')])
```

```
[ ]: let rec map2 f l1 l2 =
  match l1, l2 with
  | [], [] -> []
  | x::xs, y::ys -> f x y::map2 f xs ys
  | _, _ -> failwith "zip: lists have different lengths"
```

```
[ ]: (** [zip l1 l2] is the list of pairs of elements from lists [l1] and [l2].
      pre: [length l1 = length l2]
      *)
let zip l1 l2 =
  map2 (fun x y -> (x,y)) l1 l2
```

```
[ ]: let _ = assert (zip [1;2;3] ['a';'b';'c'] = [(1,'a');(2,'b');(3,'c')])
```

```
[ ]: (** [unzip l] is the pair of lists of the first and second elements of the
      ↪pairs in list [l].
      pre: [true]
      *)
let rec unzip l =
  match l with
  | [] -> ([], [])
  | (x,y)::xs ->
    let (xs, ys) = unzip xs in
    (x::xs, y::ys)
```

```
[ ]: let _ = assert (unzip [] = ([], []))
let _ = assert (unzip [(1,'a');(2,'b');(3,'c')] = ([1;2;3], ['a';'b';'c']))
```

```
[ ]: (** [unzip l] is the pair of lists of the first and second elements of the
      ↪pairs in list [l].
      pre: [true]
      *)
let rec unzip l =
  List.fold_right (fun (x,y) (xs, ys) -> (x::xs, y::ys)) l ([], [])
```

```
[ ]: let _ = assert (unzip [] = ([], []))
let _ = assert (unzip [(1,'a');(2,'b');(3,'c')] = ([1;2;3], ['a';'b';'c']))
let _ = assert (unzip (zip [1;2;3] ['a';'b';'c']) = ([1;2;3], ['a';'b';'c']))
```

4.4 Function composition

The map function can be used to support function composition.

```
[ ]: let rec duplicate l =
  match l with
  | [] -> []
  | x::xs -> (2*x)::duplicate xs
```

```
[ ]: let _ = assert (duplicate [1;2;3;4;5] = [2;4;6;8;10])
```

```
[ ]: let duplicate l =
  List.map (fun x -> 2*x) l
```

```
[ ]: let _ = assert (duplicate [1;2;3;4;5] = [2;4;6;8;10])
```



```
[ ]: let string_list_of_int_list l =
      List.map string_of_int l

[ ]: let _ = assert (string_list_of_int_list [1;2;3;4;5] = ["1";"2";"3";"4";"5"])

[ ]: let string_list_of_duplicate_int_list l = string_list_of_int_list (duplicate l)

[ ]: let _ = assert (string_list_of_duplicate_int_list [1;2;3;4;5] = ["2";"4";"6";
      ↪ "8";"10"])

[ ]: let compose f g x = f (g x);;

      let string_list_of_duplicate_int_list l = List.map (compose string_of_int (fun ↪
      ↪ x -> 2*x)) l

[ ]: let _ = assert (string_list_of_duplicate_int_list [1;2;3;4;5] = ["2";"4";"6";
      ↪ "8";"10"])
```

4.5 Compacting elements

```
[ ]: (** [pack l] is the list of pairs of elements that contain the number of times ↪
      ↪ an element appears in a sequence [l].
      pre: [true]
      *)
      let rec pack l =
        match l with
        | [] -> []
        | x::xs ->
          let tail = pack xs in
          begin match tail with
          | [] -> [(x, 1)]
          | (g, n)::gs -> if g = x then (g, n+1)::gs else (x, 1)::tail
          end

[ ]: pack ["a"; "a"; "b"; "c"; "c"; "c"; "d"; "e"; "e"; "f"; "f"; "f"; "f"; "g"; "h";
      ↪ "i"; "i"; "i"; "i"; "i"];;

      let _ = assert (pack ["a"; "a"; "b"; "c"; "c"; "c"; "d"; "e"; "e"; "f"; "f"; ↪
      ↪ "f"; "f"; "g"; "h"; "i"; "i"; "i"; "i"; "i"] = [("a", 2); ("b", 1); ("c", 3);
      ↪ ("d", 1); ("e", 2); ("f", 4); ("g", 1); ("h", 1); ("i", 5)])

[ ]: (** [pack l] is the list of pairs of elements that contain the number of times ↪
      ↪ an element appears in a sequence [l].
      pre: [true]
      *)
      let pack l =
        let rec f x acc =
```

```

    match acc with
    | [] -> [(x, 1)]
    | (g, n)::gs -> if x = g then (g,n+1)::gs else (x,1)::acc
in List.fold_right f l []

```

```

[ ]: pack ["a"; "a"; "b"; "c"; "c"; "c"; "d"; "e"; "e"; "f"; "f"; "f"; "f"; "g"; "h";
      ↪ "i"; "i"; "i"; "i"; "i"];;

let _ = assert (pack ["a"; "a"; "b"; "c"; "c"; "c"; "d"; "e"; "e"; "f"; "f";
      ↪ "f"; "f"; "g"; "h"; "i"; "i"; "i"; "i"; "i"] = [("a", 2); ("b", 1); ("c", 3);
      ↪ ("d", 1); ("e", 2); ("f", 4); ("g", 1); ("h", 1); ("i", 5)])

```

```

[ ]: let rec unpack l =
    match l with
    | [] -> []
    | (x, n)::xs ->
        if n = 1 then x::unpack xs
        else x::unpack ((x, n-1)::xs)

```

```

[ ]: let _ = assert (unpack [("a", 2); ("b", 1); ("c", 3); ("d", 1); ("e", 2); ("f",
      ↪ 4); ("g", 1); ("h", 1); ("i", 5)] = ["a"; "a"; "b"; "c"; "c"; "c"; "d"; "e";
      ↪ "e"; "f"; "f"; "f"; "f"; "f"; "g"; "h"; "i"; "i"; "i"; "i"; "i"]);;

let l = ["a"; "a"; "b"; "c"; "c"; "c"; "d"; "e"; "e"; "f"; "f"; "f"; "f"; "g";
      ↪ "h"; "i"; "i"; "i"; "i"; "i"] in
    assert (unpack (pack l) = l)

```

4.6 Grouping elements

```

[ ]: let rec group_by l =
    match l with
    | [] -> []
    | x::xs ->
        let gs = group_by xs in
        begin match gs with
        | [] -> [[x]]
        | g::gs ->
            begin match g with
            | [] -> assert false (* cannot be empty*)
            | y::_ -> if y = x then (x::g)::gs else [x]::g::gs
            end
        end
    end

```

```

[ ]: let _ = assert (group_by [1; 1; 2; 3; 3; 3; 4; 5; 5; 6; 6; 6; 6; 7; 8; 9; 9; 9;
      ↪ 9; 9] = [[1; 1]; [2]; [3; 3; 3]; [4]; [5; 5]; [6; 6; 6; 6]; [7]; [8]; [9; 9;
      ↪ 9; 9; 9]])

```

```
[ ]: let group_by l =
      let rec f x acc =
        match acc with
        | [] -> [[ x ]]
        | g::gs ->
          begin match g with
          | [] -> assert false (* all lists have elements, see above *)
          | y::ys -> if x = y then (x::y::ys)::gs else [ x ]::acc
          end
      in List.fold_right f l []
```

```
[ ]: let _ = assert (group_by [1; 1; 2; 3; 3; 3; 4; 5; 5; 6; 6; 6; 6; 7; 8; 9; 9; 9; 9;
↪9; 9] = [[1; 1]; [2]; [3; 3; 3]; [4]; [5; 5]; [6; 6; 6; 6]; [7]; [8]; [9; 9; 9;
↪9; 9; 9]])
```

4.7 Ungrouping elements

```
[ ]: let rec ungroup l =
      match l with
      | [] -> []
      | []::gs -> ungroup gs
      | (x::xs)::gs -> x::ungroup (xs::gs)
```

```
[ ]: let _ = assert (ungroup [[1; 1]; [2]; [3; 3; 3]; [4]; [5; 5]; [6; 6; 6; 6]; [7];
↪ [8]; [9; 9; 9; 9]] = [1; 1; 2; 3; 3; 3; 4; 5; 5; 6; 6; 6; 6; 7; 8; 9; 9; 9;
↪9; 9; 9]);;

let l = [1; 1; 2; 3; 3; 3; 4; 5; 5; 6; 6; 6; 6; 7; 8; 9; 9; 9; 9; 9] in assert_
↪(ungroup (group_by l) = l)
```

```
[ ]: let ungroup gs = List.fold_right (fun xs ys -> (List.fold_right (fun x ys -> x :
↪: ys) xs ys)) gs []
```

```
[ ]: let ungroup gs = List.fold_right (@) gs []
```

```
[ ]: let ungroup xs = List.concat xs
```

```
[ ]: let l = [1; 1; 2; 3; 3; 3; 4; 5; 5; 6; 6; 6; 6; 7; 8; 9; 9; 9; 9; 9] in assert_
↪(ungroup (group_by l) = l)
```

5 Merge of sorted lists

```
[ ]: let rec merge l1 l2 =
      match l1, l2 with
      | [], [] -> []
      | x::xs, [] -> l1
      | [], y::ys -> l2
```

```
| [], y::ys -> l2  
| x::xs, y::ys -> if x < y then x::merge xs l2 else y::merge l1 ys
```

```
[ ]: let _ = assert (merge [1;3;5;7;9] [2;4;6;8;10] = [1;2;3;4;5;6;7;8;9;10])
```